



UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura en Programación

Asignatura: Programación 3

TRABAJO PRÁCTICO INTEGRADOR: FOOD STORE

Alumno: CISNEROS LUIS

DNI: 35109356

Índice

1. Marco Teórico	3
2. Decisiones Técnicas y Arquitectura	5
3. Dificultades y Soluciones	9

1. Marco Teórico

1.1 Java 23

Java 23 es la versión del lenguaje utilizada en el backend. Entre las características aprovechadas se destacan los *switch rule expressions* (introducidos como estable en Java 14), que simplifican los menús de consola con sintaxis compacta y sin *fall-through* accidental. Java sigue el modelo orientado a objetos y fue elegido por ser el lenguaje central de la materia.

1.2 JPA / Hibernate 6

JPA (Jakarta Persistence API 3.1) es la especificación estándar de Java para el mapeo objeto-relacional (ORM). Define las anotaciones (`@Entity`, `@Table`, `@Column`, `@OneToMany`, etc.) y el ciclo de vida de las entidades, sin estar atada a ninguna implementación concreta.

Hibernate 6.4 es el proveedor JPA utilizado. Se encarga de traducir las operaciones sobre objetos Java a sentencias SQL, gestionar la caché de primer nivel (`EntityManager`) y administrar las transacciones.

Conceptos clave aplicados:

- `@MappedSuperclass`: permite que Base comparta campos comunes (id, eliminado, createdAt) con todas las entidades sin ser una entidad en sí misma.
- `@PrePersist`: callback del ciclo de vida JPA que asigna automáticamente createdAt antes del primer guardado.
- JPQL (Jakarta Persistence Query Language): lenguaje de consulta orientado a objetos, independiente del motor de base de datos. Se usa en todos los repositorios para filtrar por campos como eliminado, estado y relaciones entre entidades.
- Carga perezosa (`FetchType.LAZY`): los detalles de un pedido se cargan solo cuando se accede a la colección, evitando cargas innecesarias.
- Transacciones explícitas: cada operación que modifica datos se envuelve en `begin()` / `commit()` / `rollback()` manualmente.

1.3 H2

H2 es una base de datos relacional en Java, embebida en el proceso. Se configuró en modo **archivo** (`./data/jpa_db.mv.db`), lo que permite que los datos persistan entre ejecuciones sin necesitar un servidor de base de datos externo. Es ideal para proyectos académicos y de desarrollo local.

1.4 Lombok

Lombok es un procesador de anotaciones que genera código repetitivo en tiempo de compilación. Se utilizó para eliminar boilerplate de *getters*, *setters*, constructores y el patrón *builder* (`@Getter`, `@Setter`, `@Builder`, `@SuperBuilder`, `@NoArgsConstructor`, `@AllArgsConstructor`).

1.5 Gradle 8

Gradle es la herramienta de construcción del proyecto backend. Gestiona dependencias desde Maven Central y proporciona el comando `./gradlew run` para ejecutar la aplicación pasando `System.in` al proceso, lo que habilita la lectura de consola.

1.6 TypeScript 5

TypeScript es un superconjunto tipado de JavaScript. Aporta verificación estática de tipos en tiempo de desarrollo, lo que permite detectar errores antes de ejecutar el código. Se utilizó con la configuración estricta (`strict: true`, `noUnusedLocals`, `noUnusedParameters`).

1.7 Vite 5

Vite es el entorno de desarrollo y bundler del frontend. Provee un servidor de desarrollo con recarga en caliente (HMR) y genera el bundle de producción con Rollup. Se configuró con base: `'./'` para rutas relativas y `fs.allow: ['.']` para permitir la lectura de los archivos JSON de datos desde la carpeta `../data/`.

1.8 SPA (Single Page Application)

Una SPA carga una única página HTML y actualiza el contenido dinámicamente mediante JavaScript, sin recargas completas. La navegación se gestiona mediante el fragmento de la URL (`#/ruta`), lo que no requiere configuración de servidor para el enrutamiento.

1.9 localStorage

`localStorage` es una API del navegador que permite almacenar datos de manera persistente entre sesiones en el cliente. En este proyecto actúa como base de datos del frontend: almacena categorías, productos, usuarios, pedidos y la sesión activa.

2. Decisiones Técnicas y Arquitectura

2.1 Estructura general del proyecto

El proyecto se divide en dos módulos completamente independientes que comparten una carpeta de datos semilla:

```
Trabajo Integrador/  
├─ data/                ← JSON de datos semilla compartidos  
│   ├── categorias.json  
│   ├── productos.json  
│   ├── usuarios.json  
│   └── pedidos.json  
├─ foodstore-backend/   ← Módulo Java (consola)  
└─ foodstore-frontend/ ← Módulo TypeScript (SPA web)
```

Esta separación permite desarrollar y ejecutar cada parte de forma autónoma, reflejando una arquitectura desacoplada donde el frontend no depende del backend en ejecución.

2.2 Patrón Repository con genérico BaseRepository<T>

Se implementó el patrón *Repository* para aislar la lógica de acceso a datos de la lógica de negocio en `Main.java`. La clase `BaseRepository<T>` `extends Base<T>` concentra las operaciones CRUD comunes a todas las entidades:

Método	Descripción
<code>guardar(T entity)</code>	Alta si <code>id == null</code> , actualización con <code>merge()</code> si ya tiene <code>id</code> .
<code>buscarPorId(Long id)</code>	Retorna <code>Optional<T></code> usando <code>em.find()</code> .
<code>listarActivos()</code>	JPQL filtrando <code>eliminado = false</code> .
<code>eliminarLogico(Long id)</code>	Marca <code>eliminado = true</code> sin borrar el registro.

Cada repositorio concreto (`CategoriaRepository`, `ProductoRepository`, `UsuarioRepository`, `PedidoRepository`) hereda estas operaciones y agrega solo las consultas que le son propias:

```
// Ejemplo: buscar productos de una categoría  
"SELECT p FROM Producto p WHERE p.categoria.id = :catId AND p.eliminado = false"
```

Por qué este patrón: centraliza el manejo de `EntityManager` y transacciones en un único lugar, reduciendo la duplicación y facilitando el mantenimiento.

2.3 Soft Delete (baja lógica)

Ninguna entidad se elimina físicamente de la base de datos. La clase `Base` contiene el campo `eliminado` (booleano, por defecto `false`). Al "eliminar" un registro se llama a `eliminarLogico()`, que lo marca con `eliminado = true`.

Todas las consultas JPQL filtran `eliminado = false`, con lo que los registros dados de baja dejan de ser visibles sin perderse la trazabilidad histórica.

```
// En Base.java
@Column(name = "eliminado", nullable = false)
@Builder.Default
private boolean eliminado = false;
```

Por qué: es una práctica estándar en sistemas de gestión; preserva la integridad referencial y permite auditoría futura.

2.4 Singleton JPAUtil

JPAUtil garantiza que exista una sola instancia de `EntityManagerFactory` durante toda la vida de la aplicación. Crear la fábrica es una operación costosa (conecta con la base, lee el `persistence.xml`, inicializa el pool); crearla una vez y reutilizarla es la práctica recomendada por la especificación JPA.

```
public static EntityManagerFactory getEntityManagerFactory() {
    if (emf == null || !emf.isOpen()) {
        emf = Persistence.createEntityManagerFactory(PERSISTENCE_UNIT);
    }
    return emf;
}
```

2.5 Transacción atómica en `crearPedido()`

La creación de un pedido involucra múltiples operaciones en la misma transacción:

- Verificar que el usuario exista y no esté dado de baja.
- Para cada ítem: verificar disponibilidad y stock suficiente.
- Crear el `DetallePedido` y calcular el subtotal.
- Descontar el stock del producto.
- Persistir el pedido y vincularlo al usuario.

Si cualquier paso falla, se ejecuta `rollback()` completo, garantizando que no queden datos parciales en la base. Esto respeta las propiedades ACID de las transacciones relacionales.

```
// Descuento de stock en la misma transacción
producto.setStock(producto.getStock() - cantidad);
// El EntityManager gestionado sincroniza el cambio al hacer commit
```

2.6 Router hash SPA

El frontend implementa un router hash propio sin librerías externas. Registra rutas con `registerRoute(pattern, handler)` y resuelve el fragmento de la URL (`window.location.hash`) en cada evento `hashchange`. Soporta segmentos dinámicos con la notación `:param`:

```
/producto/:id → params = { id: "5" }
```

Por qué sin librería: el proyecto no usa ningún framework (React, Vue, Angular), por lo que incorporar un router externo añadiría dependencias sin ganancia conceptual. Un router de ~60 líneas satisface todos los casos de uso del proyecto.

2.7 Guards de navegación (auth y soloAdmin)

El acceso a las rutas está protegido por funciones *guard* de orden superior que envuelven al handler:

```
function auth(handler: RouteHandler): RouteHandler {
  return (app, params) => {
    if (!isAuthenticated()) { navigate("/login"); return; }
    handler(app, params);
  };
}

function soloAdmin(handler: RouteHandler): RouteHandler {
  return (app, params) => {
    if (!isAuthenticated()) { navigate("/login"); return; }
    if (!isAdmin()) { navigate("/store"); return; }
    handler(app, params);
  };
}
```

Esto garantiza que ningún usuario no autenticado pueda ver páginas de la tienda, y que ningún usuario sin rol ADMIN pueda acceder al panel de administración.

2.8 Datos semilla y localStorage

Al iniciar la aplicación por primera vez, `seedData()` carga los archivos JSON de la carpeta `data/` en `localStorage`. En arranques posteriores, detecta la clave `fs_seeded` y omite la carga, preservando los cambios del usuario (pedidos realizados, etc.).

Los cuatro almacenes (`fs_categorias`, `fs_productos`, `fs_usuarios`, `fs_pedidos`) se gestionan a través del módulo `storage.ts`, que expone getters y setters tipados. La sesión activa se guarda en `fs_session` sin incluir la contraseña.

2.9 Modelo de dominio

Backend

```
Base (abstracta)
├─ Categoria (id, nombre, descripcion)
├─ Producto (nombre, descripcion, precio, stock, disponible → Categoria)
├─ Usuario (nombre, mail, password, rol: ADMIN|USUARIO → [Pedido])
└─ Pedido (fecha, estado, total, formaPago → [DetallePedido])
    └─ DetallePedido (cantidad, subtotal → Producto)
```

La interfaz Calculable define el contrato `calcularTotal()`, implementado por `Pedido` para recalcular el total iterando sobre sus detalles.

Enums:

Enum	Valores
Rol	ADMIN, USUARIO
Estado	PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO
FormaPago	TARJETA, TRANSFERENCIA, EFECTIVO

2.10 Capturas de las interfaces principales

Pantalla	Descripción
Login	Formulario de autenticación con validación de credenciales
Tienda (Home)	Catálogo de productos con filtro por categoría
Detalle de producto	Imagen, descripción, precio y botón "Agregar al carrito"
Carrito	Resumen de compra con cantidades y total; confirmación de pedido
Mis pedidos	Historial del cliente con detalle y estado de cada pedido
Panel admin — Dashboard	Estadísticas generales: categorías, productos, pedidos, ingresos
Panel admin — Pedidos	Tabla con filtro por estado (dropdown)
Backend consola	Menú principal con submenús de gestión y reportes

3. Dificultades y Soluciones

3.1 Lock de la base de datos H2 al relanzar la aplicación

Problema: al ejecutar `./gradlew run` con un proceso Java previo aún activo, H2 lanzaba un error de lock (Database may be already in use) porque el archivo `jpa_db.mv.db` soporta una sola conexión en modo archivo.

Solución: identificar y cerrar el proceso Java anterior antes de relanzar. En PowerShell: `Get-Process java | Stop-Process -Force`. Para prevención, `JPAUtil.close()` se llama siempre al salir del menú principal, liberando el `EntityManagerFactory` y la conexión.

3.2 LazyInitializationException al acceder a los detalles del pedido fuera del EntityManager

Problema: los `DetallePedido` de un `Pedido` se cargan con `FetchType.LAZY`. Al cerrar el `EntityManager` antes de acceder a la colección desde `Main.java`, Hibernate lanzaba `LazyInitializationException` porque la sesión ya no estaba activa.

Solución: en `crearPedido()`, antes de cerrar el `EntityManager`, se fuerza la inicialización de la colección con una llamada a `.size()`:

```
pedido.getDetalles().size(); // fuerza la carga LAZY antes de cerrar el EM
tx.commit();
```

Esto materializa la colección en memoria mientras la sesión sigue abierta.

3.3 Warning de Hibernate 6 sobre H2Dialect

Problema: al arrancar, Hibernate imprimía "H2Dialect does not need to be specified explicitly", lo que generaba confusión sobre si la configuración era correcta.

Solución: es un *warning* benigno propio de Hibernate 6.x, que ahora detecta el dialecto automáticamente sin necesidad de declararlo en `persistence.xml`. No afecta el funcionamiento; se documentó para no invertir tiempo en resolverlo.

3.4 Acceso a los JSON de data/ desde el frontend con Vite

Problema: el frontend necesitaba importar los archivos JSON ubicados en `../data/` (fuera de `foodstore-frontend/`). Vite, por defecto, restringe el acceso a archivos fuera del directorio raíz del proyecto por seguridad.

Solución: se añadió la opción `server.fs.allow: ['.']` en `vite.config.ts`, permitiendo que Vite sirva archivos del directorio padre. Los JSON se importan como módulos TypeScript gracias a `resolveJsonModule: true` en `tsconfig.json`.

3.5 Consistencia de stock en la transacción del pedido

Problema: al crear un pedido con múltiples productos, si el descuento de stock de un producto fallaba (stock insuficiente), los productos previos ya procesados en el loop podían quedar modificados en memoria aunque la transacción no se hubiera confirmado.

Solución: toda la creación del pedido ocurre dentro de una única transacción con un solo EntityManager. Las entidades Producto recuperadas con `em.find()` son entidades gestionadas (*managed*): Hibernate rastrea sus cambios internamente y solo los sincroniza con la base al hacer `commit()`. Si se lanza una excepción en cualquier punto del loop, el `catch` ejecuta `rollback()` y ningún cambio se persiste.